

IMPORTANTE: Cree un programa en Pascal. **Utilice su apellido como nombre del programa y del archivo .pas, y guárdelo en el Escritorio de su computadora. 10/09/2024**

Una biblioteca necesita un sistema para procesar la información de los libros. De cada libro se conoce: ISBN, año de edición, código del autor y código de género (1 a 15).

- a) Implementar un módulo que lea información de los libros y retorne una estructura de datos eficiente para la búsqueda por código de autor que contenga código de autor y una lista de todos sus libros. La lectura finaliza al ingresar el valor 0 para un ISBN.
- b) Realizar un módulo que reciba la estructura generada en el inciso a) y un código. El módulo debe retornar una lista con todos los códigos de género y su cantidad de libros, para el autor asociado al código ingresado.
- c) Realizar un módulo recursivo que reciba la estructura generada en inciso b) y retorne cantidad y código de autor con mayor cantidad de libros.

NOTA: Implementar el programa principal, que invoque a los incisos a, b y c. En caso de ser necesario, puede utilizar los módulos que se encuentran a continuación.

<pre>type sub_genero = 1..15; type libro = record isbn: integer; codAutor: integer; codGenero: sub_genero; end; procedure leerLibro(var l: libro); begin read(l.isbn); if (l.isbn <> 0) then begin l.codAutor := Random(20000) + 1; l.codGenero := Random(20) + 1; end; end; procedure agregarAlVector(var V: vector; var dimL: integer; R: registro); begin if (dimL < dimF) then begin dimL := dimL + 1; V[dimL] := R; end; end; procedure agregarAdelante(var L: lista; R: registro); var nuevo: lista; begin new(nuevo); nuevo^.dato := R; nuevo^.sig := L; L := nuevo; end;</pre>	<pre>procedure agregarAtras(var L, ult: lista; registro); var nuevo: lista; begin new(nuevo); nuevo^.dato := R; nuevo^.sig := NIL; if (L = nil) then L := nuevo else ult^.sig := nuevo; ult := nuevo; end; procedure insertarOrdenado(var L: lista; registro); var nuevo, ant, act: lista; begin act := L; new(nuevo); nuevo^.dato := R; while (act <> NIL) and (act^.dato.<R. ..) begin ant := act; act := act^.sig; end; if (act = L) then L := nuevo else ant^.sig := nuevo; nuevo^.sig := act; end;</pre>
---	---

IMPORTANTE: Cree un programa en Pascal. **Utilice su apellido como nombre del programa y del archivo .pas, y guárdelo en el Escritorio de su computadora. 09/09/2024**

Una clínica necesita un sistema para el procesamiento de las atenciones realizadas a los pacientes en julio de 2024.

- a) Implementar un módulo que lea información de las atenciones. De cada atención se lee: matrícula del médico, DNI del paciente, día y diagnóstico (valor entre A y F). La lectura finaliza con DNI = 0. **Se sugiere utilizar el módulo leerAtención()**. El módulo debe retornar dos estructuras:
- i. Un árbol binario de búsqueda ordenado por matrícula del médico. Para cada matrícula de médico debe almacenarse la cantidad de atenciones realizadas.
 - ii. Un vector que almacene por cada diagnóstico posible el tipo del género y la lista de los DNI de pacientes atendidos con ese diagnóstico.
- b) Implementar un módulo que reciba el árbol generado en a) y una matrícula y retorne la cantidad total de atenciones realizadas por los médicos con matrícula superior a la matrícula ingresada.
- c) Realizar un módulo recursivo que reciba el vector generado en a) y retorne el diagnóstico con mayor cantidad de pacientes atendidos.

NOTA: Implementar el programa principal, que invoque a los incisos a, b y c. En caso de ser necesario, puede utilizar los módulos que se encuentran a continuación.

<pre>type atencion = record matricula: integer; dni: integer; dia: integer; diagnostico: char; end; var v: array [1..6] of string = ('A', 'B', 'C', 'D', 'E', 'F'); procedure leerAtencion(var a: atencion); begin a.dni := Random(5000); if (a.dni <> 0) then begin a.matricula := Random(1000) + 2000; a.dia := Random(31) + 1; a.diagnostico := v[Random(6) + 1]; end; end; procedure agregarAlVector(var V: vector; var dimL: integer; R: registro); begin if (dimL < dimF) then begin dimL := dimL + 1; V[dimL] := R; end; end; procedure agregarAdelante(var L: lista; registro); var nuevo: lista; begin new(nuevo); nuevo^.dato := R; nuevo^.sig := L; L := nuevo; end;</pre>	<pre>procedure agregarAtras(var L, ult: lista; R: registro); var nuevo: lista; begin new(nuevo); nuevo^.dato := R; nuevo^.sig := NIL; if (L = nil) then L := nuevo else ult^.sig := nuevo; ult := nuevo; end; procedure insertarOrdenado(var L: lista; R: registro); var nuevo, ant, act: lista; begin act := L; new(nuevo); nuevo^.dato := R; while (act <> NIL) and (act^.dato.<R. ..) do begin ant := act; act := act^.sig; end; if (act = L) then L := nuevo else ant^.sig := nuevo; nuevo^.sig := act; end;</pre>
--	--

Correo Argentino desea analizar la información de los paquetes enviados durante 2024. De cada paquete enviado se conoce: código de envío, DNI del emisor, DNI del receptor, cantidad de objetos en el paquete y peso del paquete en gramos.

Realice un programa que contenga e invoque a:

- a. Un módulo que lea la información de los paquetes enviados y retorne una estructura con todos los paquetes leídos. La estructura debe ser eficiente para buscar **por peso del paquete en gramos**. La lectura finaliza al leer un paquete con código de envío **0**.
- b. Un módulo que reciba la estructura generada en a) y dos valores, y retorne una lista con todos los paquetes enviados cuyo peso esté entre los dos valores recibidos.
- c. Un módulo que reciba la estructura generada en a) y retorne toda la información del paquete enviado con mayor cantidad de objetos.

En caso de ser necesario, adjuntamos porciones de código que puede utilizar en su programa.

<pre>Type paquete = record codEnvio: integer; DNIemisor: integer; DNIreceptor: integer; cantObjetos: integer; pesoGr: integer; end; procedure leerPaquete(var p: paquete); { Para agilizar la carga, a todos los campos del registro se les asignan valores al azar } begin p.codEnvio := random(101); if (p.codEnvio <> 0) then begin p.DNIemisor := 9 + random(50); p.DNIreceptor:= 6 + random(50); p.cantObjetos:= random(10); p.pesoGr := 100 + random(1000); end; end; procedure agregarAtras(var L, ult: lista; R: registro); var nuevo: lista; begin new(nuevo); nuevo^.dato := R; nuevo^.sig := NIL; if (L = nil) then L := nuevo else ult^.sig := nuevo; ult := nuevo; end;</pre>	<pre>procedure agregarAdelante(var L: lista; R: registro); var nuevo: lista; begin new(nuevo); nuevo^.dato := R; nuevo^.sig := L; L := nuevo; end; procedure insertarOrdenado(var L: lista; R: registro); var nuevo, ant, act: lista; begin act := L; new(nuevo); nuevo^.dato := R; while (act <> NIL) and (act^.dato.campoOrden < R.campoOrden) do begin ant := act; act := act^.sig; end; if (act = L) then L := nuevo else ant^.sig := nuevo; nuevo^.sig := act; end;</pre>
---	--

La Feria del Artesano necesita un sistema para obtener estadísticas sobre las artesanías presentadas.

a) Implementar un módulo que lea información de las artesanías. De cada artesanía se conoce: número de identificación de la artesanía, DNI del artesano y código del material base (1: madera, 2: yeso, 3: cerámica, 4: vidrio, 5: acero, 6: porcelana, 7: lana, 8: cartón). La lectura finaliza con el valor 0 para el DNI. **Se sugiere utilizar el módulo leerArtesanía()**. El módulo debe retornar dos estructuras:

i. Un árbol binario de búsqueda ordenado por DNI del artesano. Para cada DNI del artesano debe almacenarse la cantidad de artesanías correspondientes al DNI.

ii. Un vector que almacene para cada material base, el código del material base y la cantidad de artesanías del material base.

b) Implementar un módulo que reciba el árbol generado en a) y un DNI. El módulo debe retornar la cantidad de artesanos con DNI menor al DNI ingresado.

c) Implementar un módulo que reciba el vector generado en a) y lo ordene por cantidad de artesanías de menor a mayor y retorne el nombre del material base con mayor cantidad de artesanías.

NOTA: Implementar el programa principal, que invoque a los incisos a, b y c. En caso de ser necesario, puede utilizar los módulos que se encuentran a continuación.

```
type subMaterial = 1..8;

type
  artesania = record
    codArtesania: integer;
    DNI: integer;
    material: subMaterial;
  end;

var v: array [1..8] of string =
  ('madera', 'yeso', 'ceramica', 'vidrio',
  'acero', 'porcelana', 'lana', 'carton');
```

```
procedure leerArtesania(var a: artesania);
begin
  a.DNI := Random(10000);
  if (a.DNI <> 0) then begin
    a.codArtesania := Random(100) + 2000;
    a.material := Random(8) + 1;
  end;
end;
```

```
procedure agregarAlVector(var V: vector; var
dimL: integer; R: registro);
begin
  if (dimL < dimF) then begin
    dimL := dimL + 1;
    V[dimL] := R;
  end;
end;
```

```
procedure agregarAdelante(var L: lista; R:
registro);
var
  nuevo: lista;
begin
  new(nuevo);
  nuevo^.dato := R;
  nuevo^.sig := L;
  L := nuevo;
end;
```

```
procedure agregarAtras(var L, ult: lista; R:
registro);
var
  nuevo: lista;
begin
  new(nuevo);
  nuevo^.dato := R;
  nuevo^.sig := NIL;
  if (L = nil) then
    L := nuevo
  else
    ult^.sig := nuevo;
    ult := nuevo;
end;
```

```
procedure insertarOrdenado(var L: lista; R:
registro);
var
  nuevo, ant, act: lista;
begin
  act := L;
  new(nuevo);
  nuevo^.dato := R;
  while (act <> NIL) and (act^.dato.<R. ..) do
  begin
    ant := act;
    act := act^.sig;
  end;
  if (act = L) then
    L := nuevo
  else
    ant^.sig := nuevo;
    nuevo^.sig := act;
  end;
```

